Sentiment Analysis — Project Overview

Cross-lingual sentiment analysis for Dravidian languages

`com.sentiment:sentiment-analysis:0.0.1-SNAPSHOT`

TABLE OF CONTENTS

1. Introduction
2. Project at a Glance
3. Technology Stack
4. ML Model
5. Architecture
6. Project Structure
7. Features
8. Security
9. Database
10. API Endpoints
11. Testing
12. Configuration
13. Building & Running

1. Introduction

The **Sentiment Analysis** application is a full-stack web application that determines whether text expresses a **positive**, **negative**, or **neutral** sentiment. It is designed specifically for **Dravidian languages** — Tamil, Malayalam, Kannada, and Telugu — and supports both native script and romanized (Latin-letter) input.

The system uses a pre-trained **XLM-RoBERTa** transformer model running entirely inside the JVM via ONNX Runtime, requiring no external Python service or GPU at runtime. Romanized text is automatically transliterated to native script using the **ICU4J** library before inference, dramatically improving accuracy for Latin-script input.

2. Project at a Glance

38	7	44
JAVA SOURCE FILES	TEST CLASSES	TEST CASES
~5,000	**4**	**6**
LINES OF JAVA	LANGUAGES SUPPORTED	HTML TEMPLATES

3. Technology Stack

CATEGORY	TECHNOLOGY	VERSION
Language	Java	21
Framework	Spring Boot	3.5.11
Build Tool	Maven	3.x (wrapper included)
Web / MVC	Spring Web + Thymeleaf	—
Security	Spring Security 6	—
ORM	Spring Data JPA / Hibernate	—
Default Database	H2 (file-based)	—
Optional Database	MySQL (via `mysql` profile)	—

CATEGORY	TECHNOLOGY	VERSION
ML Inference	DJL (Deep Java Library) + ONNX Runtime	0.36.0
Transliteration	ICU4J	75.1
Validation	Jakarta Bean Validation	—
Frontend	Vanilla HTML/CSS/JS (no framework)	—

4. ML Model

Sentiment Model Details

MODEL NAME	`cardiffnlp/twitter-xlm-roberta-base-sentiment`
ARCHITECTURE	XLMRobertaForSequenceClassification
BASE MODEL	XLM-RoBERTa Base (multilingual, 100+ languages)
LAYERS	12 transformer layers, 768 hidden units, 12 attention heads
OUTPUT LABELS	Negative (0) • Neutral (1) • Positive (2)
RUNTIME FORMAT	ONNX (exported via `python/export_model.py`)
INFERENCE ENGINE	DJL 0.36.0 + ONNX Runtime (CPU)
MAX SEQUENCE LENGTH	256 tokens
LOW CONFIDENCE THRESHOLD	60%
MODEL LOCATION	`./models/xlmr-sentiment/`

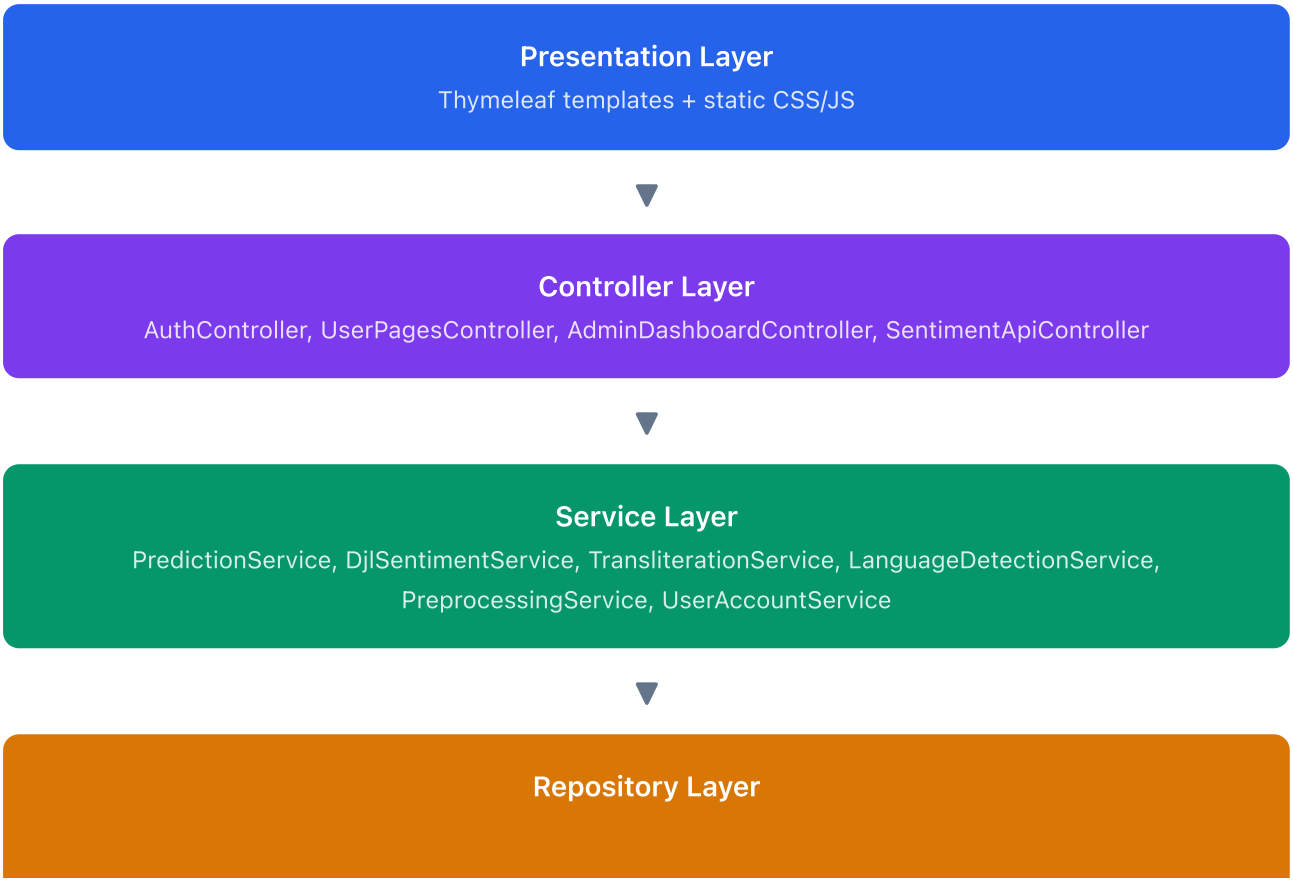
Inference Pipeline

The end-to-end prediction pipeline works as follows:

STEP	COMPONENT	ACTION
1	`PreprocessingService`	Cleans and normalizes input text
2	`LanguageDetectionService`	Detects the language and script type (native vs. Latin)
3	`TransliterationService`	If Latin script detected, transliterates to native script via ICU4J
4	`SentimentTranslator`	Tokenizes text with XLM-RoBERTa tokenizer
5	`DjlSentimentService`	Runs ONNX model inference, returns softmax probabilities
6	`PredictionService`	Orchestrates the pipeline, saves result to database

5. Architecture

The application follows a **layered architecture** pattern typical of Spring Boot applications:



AppUserRepository, PredictionRecordRepository (Spring Data JPA)

▼

Database
H2 (default, file-based) or MySQL (via profile)

Package Structure

PACKAGE	PURPOSE	CLASSES
`config`	Security configuration, model properties, data initialization	3
`controller`	MVC controllers for auth, user pages, admin pages	4
`controller.api`	REST API for sentiment analysis	1
`dto`	Data transfer objects for requests/responses	7
`exception`	Global exception handler, custom exceptions	3
`model.entity`	JPA entities (AppUser, PredictionRecord)	2
`model.enums`	Enumerations (Role, Sentiment)	2
`repository`	Spring Data JPA repositories	2
`security`	UserDetailsService, login success handler	2
`service`	Business logic: ML inference, transliteration, preprocessing, prediction orchestration	7
`startup`	Model readiness check on boot	1
`util`	Math utilities (softmax)	1

6. Project Structure

```
sentiment-app/
├── pom.xml
├── models/
│   └── xlmr-sentiment/          ← ONNX model + tokenizer files
├── python/
│   └── export_model.py          ← Hugging Face → ONNX export script
├── data/
│   └── sentimentdb.*            ← H2 database files (auto-created)
├── document/
│   ├── project-overview.html    ← This document
│   ├── database-architecture.*  ← Database design & ER diagram
│   ├── user-manual.*            ← End-user manual with screenshots
│   ├── dfd-sentiment-analysis.* ← Data flow diagram
│   └── screenshots/             ← Application screenshots
├── src/main/
│   ├── java/com/sentiment/analysis/
│   │   ├── SentimentApplication.java
│   │   ├── config/              ← SecurityConfig, ModelProperties, DataInitiali
│   │   ├── controller/          ← AuthController, user & admin controllers
│   │   ├── dto/                 ← Request/response objects
│   │   ├── exception/           ← Global handler, custom exceptions
│   │   ├── model/               ← JPA entities & enums
│   │   ├── repository/          ← Spring Data JPA repositories
│   │   ├── security/            ← UserDetailsService, LoginSuccessHandler
│   │   ├── service/             ← ML inference, transliteration, prediction
│   │   ├── startup/             ← Model startup check
│   │   └── util/                ← MathUtils
│   └── resources/
│       ├── application.yml          ← Main config (H2 default)
│       ├── application-mysql.yml    ← MySQL profile config
│       ├── static/
│       │   ├── css/app.css
│       │   ├── js/common.js
│       │   └── favicon.svg
│       └── templates/
│           ├── fragments/base.html  ← Shared nav, head, user menu
│           ├── auth/                ← login.html, register.html
│           ├── user/                ← dashboard, analyze, history
│           └── admin/               ← dashboard.html
└── src/test/
    └── java/...                     ← 7 test classes, 44 tests
```

7. Features

🌐 Cross-lingual Sentiment

Analyzes text in Tamil, Malayalam, Kannada, and Telugu using XLM-RoBERTa.

🔄 Auto-transliteration

Romanized Latin text is automatically converted to native script via ICU4J before analysis.

📈 Confidence Scoring

Shows per-class probabilities and flags low-confidence (<60%) predictions.

📚 Analysis History

All predictions saved with filtering by language, pagination, and timestamps.

👥 User Management

Registration, login, role-based access (USER and ADMIN roles).

🛠️ Admin Dashboard

Overview of users, model health status, and recent registrations.

🔎 Language Detection

Automatic detection of input language and script type (native vs. Latin).

⚡ In-JVM Inference

ONNX model runs inside the JVM — no Python, no GPU, no external service.

8. Security

FEATURE	IMPLEMENTATION
Authentication	Spring Security form-based login with BCrypt password hashing
Authorization	Role-based access: `USER` and `ADMIN` roles
CSRF Protection	Enabled by default on all endpoints (except H2 console)
Session Management	Server-side sessions via Spring Security
Password Validation	Minimum 8 characters, confirmed at registration

FEATURE	IMPLEMENTATION
Data Isolation	Users can only view their own analysis history

URL Access Control

URL PATTERN	ACCESS
`/login`, `/register`, `/error`	**Public**
`/css/**`, `/js/**`	**Public**
`/h2-console/**`	**Public** (dev only)
`/user/**`, `/api/sentiment/**`	**USER**
`/admin/**`, `/api/admin/**`	**ADMIN**

9. Database

The application supports two database backends:

DATABASE	PROFILE	USE CASE
H2 (file-based)	**default**	Local development and standalone deployment. Data persisted at `./data/sentimentdb`. H2 console available at `/h2-console`.
MySQL	**mysql**	Production deployment. Activate with `--spring.profiles.active=mysql`.

Database Tables

TABLE	COLUMNS	PURPOSE
`app_user`	8	User accounts: id, username, email, password (BCrypt), role, enabled, created_at, last_login

TABLE	COLUMNS	PURPOSE
`prediction_record`	13	Analysis history: id, original_text, processed_text, language, script_type, sentiment, confidence, probabilities (neg/neu/pos), transliterated, code_mixed, user_id, created_at

ⓘ See database-architecture.html for the full ER diagram, column-level details, and relationship mapping.

10. API Endpoints

Page Routes (Thymeleaf)

METHOD	URL	ROLE	DESCRIPTION
GET	`/login`	Public	Login page
GET	`/register`	Public	Registration page
POST	`/register`	Public	Process registration
GET	`/user/dashboard`	USER	User dashboard
GET	`/user/analyze`	USER	Analyze page
GET	`/user/history`	USER	Analysis history (paginated)
GET	`/admin/dashboard`	ADMIN	Admin overview

REST API

METHOD	URL	ROLE	DESCRIPTION
POST	`/api/sentiment/analyze`	USER	Run sentiment analysis on text

Analyze Request / Response

```
{
  // Request
  "text": "ഈ സിനിമ വളരെ നല്ലതാണ്",
  "language": "ml"           // optional: "ta", "ml", "kn", "te"
}

{
  // Response
  "sentiment": "POSITIVE",
  "confidence": 0.846,
  "probabilities": { "negative": 0.05, "neutral": 0.10, "positive": 0.85 },
  "language": { "code": "ml", "name": "Malayalam", "script": "MALAYALAM" },
  "transliterated": false,
  "codeMixed": false,
  "latencyMs": 142
}
```

11. Testing

TEST CLASS	SCOPE
`DjlSentimentServiceTest`	ONNX model loading, inference, softmax output
`LanguageDetectionServiceTest`	Script detection for Tamil, Malayalam, Kannada, Telugu, Latin
`TransliterationServiceTest`	ICU4J Latin-to-native script conversion for all 4 languages
`PreprocessingServiceTest`	Text cleanup, normalization, whitespace handling
`MathUtilsTest`	Softmax function correctness
`SentimentApiTest`	REST API integration tests (authenticated requests)
`SecurityAccessTest`	URL access control, role-based authorization

```
# Run all tests
./mvnw test
```

```
# Run a specific test class
./mvnw test -Dtest=DjlSentimentServiceTest
```

✓ Tests use an in-memory H2 database (`application-test.yml`) and do not require MySQL or the ONNX model files to be present for unit tests.

12. Configuration

Key Properties (`application.yml`)

PROPERTY	DEFAULT	DESCRIPTION
`server.port`	`8080`	HTTP port
`spring.datasource.url`	`jdbc:h2:file:./data/sentimentdb`	Database connection URL
`spring.jpa.hibernate.ddl-auto`	`update`	Schema auto-update on startup
`app.model.path`	`./models/xlmr-sentiment`	Path to ONNX model directory
`app.model.max-length`	`256`	Max token sequence length
`app.model.low-confidence-threshold`	`0.60`	Below this, results are flagged
`app.model.startup-check`	`true`	Verify model loads on startup

PROPERTY	DEFAULT	DESCRIPTION
`app.admin.username`	`admin`	Default admin username
`app.admin.email`	`admin@example.com`	Default admin email
`app.admin.password`	`${ADMIN_PASSWORD:Admin@12345}`	Admin password (env-overridable)

Environment Variables

VARIABLE	USED BY	PURPOSE
`ADMIN_PASSWORD`	Default profile	Override the default admin password
`DB_HOST`	MySQL profile	MySQL host (default: `localhost`)
`DB_PORT`	MySQL profile	MySQL port (default: `3306`)
`DB_NAME`	MySQL profile	Database name (default: `sentimentdb`)
`DB_USERNAME`	MySQL profile	MySQL username (default: `root`)
`DB_PASSWORD`	MySQL profile	MySQL password (default: `root`)

13. Building & Running

Prerequisites

REQUIREMENT	NOTES
Java 21+	JDK required for building; JRE sufficient for running the JAR

REQUIREMENT	NOTES
ONNX model files	Place in `./models/xlmr-sentiment/` (run `python/export_model.py` to generate)

Commands

```
# Build the project
./mvnw clean package -DskipTests

# Run with H2 (default)
java -jar target/sentiment-analysis-0.0.1-SNAPSHOT.jar

# Run with MySQL
java -jar target/sentiment-analysis-0.0.1-SNAPSHOT.jar \
  --spring.profiles.active=mysql

# Run in development mode
./mvnw spring-boot:run
```

First Launch

On first startup, the application will:

1. Create the database schema automatically (H2 file or MySQL tables)
2. Create the default admin account (`admin` / `Admin@12345`)
3. Load and verify the ONNX sentiment model
4. Start listening on `http://localhost:8080`

> ⓘ The H2 console is available at `http://localhost:8080/h2-console` for inspecting the database during development. Connect with JDBC URL `jdbc:h2:file:./data/sentimentdb` , username `sa` , no password.

Sentiment Analysis Application — Cross-lingual Dravidian Language Sentiment Analyzer

Java 21 • Spring Boot 3.5.11 • DJL 0.36.0 / ONNX Runtime • ICU4J 75.1 • XLM-RoBERTa